```
import io
import os
import calendar
import secrets
import hashlib
import re
import difflib
import unicodedata
from datetime import datetime, date
from zoneinfo import ZoneInfo

import pandas as pd
import psycopg2
import psycopg2.extras
import streamlit as st
from PIL import Image, ImageOps

FMT = "%Y-%m-%d %H:%M:%S"
HODINOVA, METROVA = "hodinová", "metrová"
TZ = ZoneInfo("Europe/Prague")

st.set_page_config(page_title="Docházka", page_icon="🏗️", layout="centered")

st.markdown(
    """
    <style>
    #MainMenu, footer {visibility: hidden;}
    header[data-testid="stHeader"] {background: transparent;}
    .block-container {padding-top: 2rem; max-width: 900px;}
    h1, h2, h3 {color: #0F2A4A; letter-spacing: -0.01em;}
    div[data-testid="stForm"] {
        background: #FFFFFF; border: 1px solid #E5E7EB; border-radius: 14px;
        padding: 1.2rem; box-shadow: 0 1px 3px rgba(15, 42, 74, 0.08);
    }
    div[data-testid="stMetric"] {
        background: #FFFFFF; border: 1px solid #E5E7EB; border-radius: 12px;
        padding: 0.8rem 1rem; box-shadow: 0 1px 3px rgba(15, 42, 74, 0.06);
    }
    .stButton > button, .stDownloadButton > button, .stFormSubmitButton > button
        border-radius: 10px; font-weight: 600; padding: 0.6rem 1rem;
    }
    button[data-baseweb="tab"] {font-weight: 600;}
    div[data-testid="stDataFrame"], div[data-testid="stDataEditor"] {
```

```
        border: 1px solid #E5E7EB; border-radius: 12px; overflow: hidden;
    }
    @media (max-width: 640px) {
        .block-container {padding: 1rem 0.8rem 3rem 0.8rem;}
        .stButton > button, .stDownloadButton > button, .stFormSubmitButton > but
            min-height: 3.2rem; font-size: 1.05rem; width: 100%;
        }
        input, textarea {font-size: 16px !important;}
        div[data-testid="stForm"] {padding: 0.9rem;}
        button[data-baseweb="tab"] {padding: 0.6rem 0.7rem; font-size: 0.95rem;}
    }
    </style>
    """,
    unsafe_allow_html=True,
)


def now():
    return datetime.now(TZ).replace(tzinfo=None)


def nn(x):
    """NaN -> None, aby se do databáze neuložilo 'NaN'."""
    if x is None:
        return None
    try:
        if pd.isna(x):
            return None
    except (TypeError, ValueError):
        pass
    return x


NOVE = "+ Jiné / nové místo…"
REZIMY = ["hodiny", "metry", "hodiny i metry"]


def norm(s):
    """Název pro porovnání: bez velikosti písmen, diakritiky, interpunkce a přeby
    s = unicodedata.normalize("NFKD", s or "")
    s = "".join(ch for ch in s if not unicodedata.combining(ch))
    s = re.sub(r"[^\w\s]", " ", s.lower())
    return " ".join(s.split())


def cisty_nazev(s):
```

```
    s = " ".join((s or "").split())
    return s[:1].upper() + s[1:]


CZ_DNY = ["po", "út", "st", "čt", "pá", "so", "ne"]


def zmensi_foto(soubor):
    """Zmenší fotku (max 1280 px, JPEG), aby zabírala ~150 kB místo několika MB."
    img = Image.open(soubor)
    img = ImageOps.exif_transpose(img).convert("RGB")
    img.thumbnail((1280, 1280))
    buf = io.BytesIO()
    img.save(buf, "JPEG", quality=70, optimize=True)
    return buf.getvalue()


# ---------- databáze (Supabase / PostgreSQL) ----------
def db_url():
    try:
        return st.secrets["DATABASE_URL"]
    except Exception:
        return os.environ.get("DATABASE_URL", "")


def kontrola_adresy(url):
    """Vrátí českou nápovědu, pokud je adresa databáze zjevně špatně."""
    u = url.strip()
    if not u:
        return "Chybí DATABASE_URL v nastavení Secrets."
    if "tvoje" in u.lower() or "TVOJE" in u:
        return "V adrese zůstal ukázkový text (tvoje-adresa / TVOJE-HESLO). Nahra
    if "YOUR-PASSWORD" in u or "[" in u or "]" in u:
        return "V adrese zůstal text [YOUR-PASSWORD] nebo hranaté závorky. Nahraď
    if not (u.startswith("postgresql://") or u.startswith("postgres://")):
        return "Adresa musí začínat postgresql://"
    if u.count("@") != 1:
        return "Adresa obsahuje víc znaků @. Heslo asi obsahuje speciální znak. N
    if "pooler.supabase.com" not in u and "supabase" in u:
        return "Není to adresa ze Session pooleru (chybí pooler.supabase.com). Zk
    return None


def connect():
    url = db_url()
    chyba = kontrola_adresy(url)
```

```
    if chyba:
        st.error("Problém s adresou databáze: " + chyba)
        st.stop()
    try:
        return psycopg2.connect(url.strip(), sslmode="require", connect_timeout=1
    except psycopg2.OperationalError as e:
        text = str(e).replace(url.strip(), "***")
        st.error("Nepodařilo se připojit k databázi. Důvod je níže, pošli mi ho (
        st.code(text)
        st.stop()


def run(sql, params=()):
    c = connect()
    try:
        with c:
            with c.cursor() as cur:
                cur.execute(sql, params)
    finally:
        c.close()


def run_many(stmts):
    c = connect()
    try:
        with c:
            with c.cursor() as cur:
                for sql, params in stmts:
                    cur.execute(sql, params)
    finally:
        c.close()


def one(sql, params=()):
    c = connect()
    try:
        with c:
            with c.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
                cur.execute(sql, params)
                r = cur.fetchone()
                return dict(r) if r else None
    finally:
        c.close()


def q(sql, params=()):
```

```
    c = connect()
    try:
        with c:
            with c.cursor() as cur:
                cur.execute(sql, params)
                rows = cur.fetchall()
                cols = [d[0] for d in cur.description]
        return pd.DataFrame(rows, columns=cols)
    finally:
        c.close()


@st.cache_resource
def init_db():
    run(
        """
        CREATE TABLE IF NOT EXISTS users(
            id SERIAL PRIMARY KEY,
            jmeno TEXT UNIQUE NOT NULL,
            heslo_hash TEXT NOT NULL,
            sul TEXT NOT NULL,
            role TEXT NOT NULL DEFAULT 'zamestnanec',
            aktivni INTEGER NOT NULL DEFAULT 1
        );
        CREATE TABLE IF NOT EXISTS zakazky(
            id SERIAL PRIMARY KEY,
            nazev TEXT UNIQUE NOT NULL,
            klient TEXT,
            typ_sazby TEXT,
            sazba DOUBLE PRECISION NOT NULL DEFAULT 0,
            aktivni INTEGER NOT NULL DEFAULT 1
        );
        CREATE TABLE IF NOT EXISTS prirazeni(
            user_id INTEGER NOT NULL,
            zakazka_id INTEGER NOT NULL,
            sazba DOUBLE PRECISION,
            PRIMARY KEY(user_id, zakazka_id)
        );
        CREATE TABLE IF NOT EXISTS dochazka(
            id SERIAL PRIMARY KEY,
            user_id INTEGER NOT NULL,
            zakazka_id INTEGER NOT NULL,
            datum TEXT NOT NULL,
            prichod TEXT,
            odchod TEXT,
            hodiny DOUBLE PRECISION,
```

```
        metry DOUBLE PRECISION,
        poznamka TEXT,
        schvaleno INTEGER NOT NULL DEFAULT 0
    );
    CREATE TABLE IF NOT EXISTS fotky(
        id SERIAL PRIMARY KEY,
        user_id INTEGER NOT NULL,
        zakazka_id INTEGER NOT NULL,
        datum TEXT NOT NULL,
        popis TEXT,
        data BYTEA NOT NULL
    );
    CREATE TABLE IF NOT EXISTS zalohy(
        id SERIAL PRIMARY KEY,
        user_id INTEGER NOT NULL,
        datum TEXT NOT NULL,
        castka DOUBLE PRECISION NOT NULL,
        poznamka TEXT
    );
    -- nový model: sazby u zaměstnance, zakázky vznikají samy
    ALTER TABLE users ADD COLUMN IF NOT EXISTS sazba_hod DOUBLE PRECISION NOT
    ALTER TABLE users ADD COLUMN IF NOT EXISTS sazba_m DOUBLE PRECISION NOT N
    ALTER TABLE zakazky ADD COLUMN IF NOT EXISTS nazev_norm TEXT;
    ALTER TABLE zakazky ADD COLUMN IF NOT EXISTS nova INTEGER NOT NULL DEFAUL
    ALTER TABLE zakazky ALTER COLUMN typ_sazby DROP NOT NULL;
    ALTER TABLE dochazka ADD COLUMN IF NOT EXISTS sazba_hod DOUBLE PRECISION;
    ALTER TABLE dochazka ADD COLUMN IF NOT EXISTS sazba_m DOUBLE PRECISION;
    """
)

# normalizované názvy starých zakázek
stare = q("SELECT id, nazev FROM zakazky WHERE nazev_norm IS NULL")
if not stare.empty:
    run_many([
        ("UPDATE zakazky SET nazev_norm=%s WHERE id=%s", (norm(r.nazev), int(
        for r in stare.itertuples()
    ])

# převod starých záznamů: sazba se zapíše přímo do záznamu (staré ceny zůstan
if one("SELECT COUNT(*) AS n FROM dochazka WHERE sazba_hod IS NULL")["n"] > 0
    run(
        """
        UPDATE dochazka d SET
          sazba_hod = CASE WHEN z.typ_sazby = %s THEN 0 ELSE COALESCE(
              (SELECT p.sazba FROM prirazeni p WHERE p.user_id=d.user_id AND
              z.sazba, 0) END,
```

```
              sazba_m = CASE WHEN z.typ_sazby = %s THEN COALESCE(
                  (SELECT p.sazba FROM prirazeni p WHERE p.user_id=d.user_id AND
                  z.sazba, 0) ELSE 0 END
              FROM zakazky z
              WHERE z.id = d.zakazka_id AND d.sazba_hod IS NULL
              """,
              (METROVA, METROVA),
          )
          # zaměstnanci dostanou sazby podle posledního záznamu
          run(
              """
              UPDATE users u SET
                sazba_hod = COALESCE((SELECT d.sazba_hod FROM dochazka d
                     WHERE d.user_id=u.id AND d.sazba_hod > 0 ORDER BY d.id DESC LI
                sazba_m = COALESCE((SELECT d.sazba_m FROM dochazka d
                     WHERE d.user_id=u.id AND d.sazba_m > 0 ORDER BY d.id DESC LIMI
              WHERE u.sazba_hod = 0 AND u.sazba_m = 0
              """
          )
      return True


# ---------- hesla ----------
def hash_pw(pw, salt):
      return hashlib.pbkdf2_hmac("sha256", pw.encode(), bytes.fromhex(salt), 200_00


def create_user(jmeno, heslo, role, sazba_hod=0.0, sazba_m=0.0):
      salt = secrets.token_hex(16)
      run(
          "INSERT INTO users(jmeno, heslo_hash, sul, role, sazba_hod, sazba_m) VALU
          (jmeno.strip(), hash_pw(heslo, salt), salt, role, float(sazba_hod), float
      )


def set_password(user_id, heslo):
      salt = secrets.token_hex(16)
      run("UPDATE users SET heslo_hash=%s, sul=%s WHERE id=%s", (hash_pw(heslo, sal


def login(jmeno, heslo):
      u = one("SELECT * FROM users WHERE jmeno=%s AND aktivni=1", (jmeno.strip(),))
      if u and secrets.compare_digest(u["heslo_hash"], hash_pw(heslo, u["sul"])):
          return u
      return None
```

```
# ---------- přihlášení ----------
def page_login():
    st.title("🏗️ Docházka")
    n = one("SELECT COUNT(*) AS n FROM users")["n"]

    if n == 0:
        st.info("První spuštění: vytvoř účet majitele (admina).")
        with st.form("setup"):
            j = st.text_input("Jméno admina")
            h1 = st.text_input("Heslo", type="password")
            h2 = st.text_input("Heslo znovu", type="password")
            if st.form_submit_button("Vytvořit"):
                if not j or len(h1) < 6:
                    st.error("Zadej jméno a heslo (min. 6 znaků).")
                elif h1 != h2:
                    st.error("Hesla se neshodují.")
                else:
                    create_user(j, h1, "admin")
                    st.success("Hotovo, teď se přihlas.")
                    st.rerun()
        return

    with st.form("login"):
        j = st.text_input("Jméno")
        h = st.text_input("Heslo", type="password")
        if st.form_submit_button("Přihlásit", use_container_width=True):
            u = login(j, h)
            if u:
                st.session_state.user = u
                st.rerun()
            else:
                st.error("Špatné jméno nebo heslo.")


# ---------- zakázky se zakládají samy ----------
def vyber_mista(uid, prefix):
    """Výběr místa. Vrací (zakazka_id, nazev_nove_zakazky). Zakázka se založí až
    zak = q(
        """SELECT z.id, z.nazev FROM zakazky z
           LEFT JOIN (SELECT zakazka_id, MAX(id) AS posledni FROM dochazka
                      WHERE user_id=%s GROUP BY zakazka_id) x ON x.zakazka_id=z.i
           WHERE z.aktivni=1
           ORDER BY x.posledni DESC NULLS LAST, z.nazev""",
        (uid,),
    )
```

```
    vse = q("SELECT id, nazev, nazev_norm FROM zakazky")

    moznosti = list(zak["nazev"]) + [NOVE]
    v = st.selectbox("Místo / zakázka", moznosti, key=f"{prefix}_misto")
    if v != NOVE:
        return int(zak.loc[zak["nazev"] == v, "id"].iloc[0]), None

    txt = st.text_input("Napiš místo (např. Nepomuk)", key=f"{prefix}_nove", max_
    n = norm(txt)
    if len(n) < 2:
        return None, None
    nazev = cisty_nazev(txt)

    shoda = vse[vse["nazev_norm"] == n]
    if not shoda.empty:
        r = shoda.iloc[0]
        st.caption(f"Zakázka **{r['nazev']}** už existuje, použije se ta.")
        return int(r["id"]), None

    normy = vse["nazev_norm"].fillna("").tolist()
    kand = difflib.get_close_matches(n, normy, n=1, cutoff=0.8)
    if not kand:
        for m in normy:
            if len(m) >= 4 and len(n) >= 4 and (m in n or n in m):
                kand = [m]
                break
    if kand:
        r = vse[vse["nazev_norm"] == kand[0]].iloc[0]
        ano = f"Ano, {r['nazev']}"
        ne = f"Ne, je to nové místo „{nazev}“"
        volba = st.radio(f"Myslíš „{r['nazev']}“?", [ano, ne], index=None, key=f"
        if volba is None:
            st.caption("Vyber jednu z možností.")
            return None, None
        if volba == ano:
            return int(r["id"]), None
    return None, nazev


def zakazka_id(zid, novy):
    """Vrátí id zakázky; když ještě neexistuje, založí ji (admin ji uvidí jako no
    if zid:
        return int(zid)
    n = norm(novy)
    r = one("SELECT id, aktivni FROM zakazky WHERE nazev_norm=%s", (n,))
    if r:
```

```
        if not r["aktivni"]:
            run("UPDATE zakazky SET aktivni=1, nova=1 WHERE id=%s", (r["id"],))
        return int(r["id"])
    try:
        r = one(
            "INSERT INTO zakazky(nazev, nazev_norm, sazba, nova) VALUES(%s,%s,0,1
            (novy, n),
        )
        return int(r["id"])
    except psycopg2.IntegrityError:
        r = one("SELECT id FROM zakazky WHERE nazev=%s", (novy,))
        return int(r["id"])


def reset_mista(prefix):
    for k in ("misto", "nove", "podobne"):
        st.session_state.pop(f"{prefix}_{k}", None)


# ---------- sazby ----------
def cur_user(uid):
    return one("SELECT id, sazba_hod, sazba_m FROM users WHERE id=%s", (int(uid),


def vyber_rezim(u, key):
    """Podle toho, jaké sazby zaměstnanec má, se zeptá, co se mu platí."""
    sh, sm = float(u["sazba_hod"] or 0), float(u["sazba_m"] or 0)
    if sh > 0 and sm > 0:
        return st.radio("Jak se to platí?", REZIMY, horizontal=True, key=key)
    if sm > 0:
        return "metry"
    return "hodiny"


def sazby(u, rezim):
    sh, sm = float(u["sazba_hod"] or 0), float(u["sazba_m"] or 0)
    return (sh if rezim in ("hodiny", "hodiny i metry") else 0.0,
            sm if rezim in ("metry", "hodiny i metry") else 0.0)


def chyba_vstupu(rezim, hodiny, metry):
    if rezim in ("hodiny", "hodiny i metry") and hodiny <= 0:
        return "Zadej hodiny."
    if rezim in ("metry", "hodiny i metry") and metry <= 0:
        return "Zadej metry."
    return None
```

```
# ---------- zaměstnanec ----------
def page_zamestnanec(user):
    uid = user["id"]
    u = cur_user(uid)

    tab1, tab2, tab3, tab4 = st.tabs(["⏱️ Dnes", "✍️ Ruční zápis", "📋 Moje zázn

    with tab1:
        otevreny = one(
            """SELECT d.*, z.nazev FROM dochazka d
               JOIN zakazky z ON z.id=d.zakazka_id
               WHERE d.user_id=%s AND d.odchod IS NULL AND d.prichod IS NOT NULL
               ORDER BY d.id DESC LIMIT 1""",
            (uid,),
        )

        if otevreny:
            start = datetime.strptime(otevreny["prichod"], FMT)
            st.success(f"Jsi na zakázce **{otevreny['nazev']}** od {start:%H:%M}"
            celkem = round((now() - start).total_seconds() / 3600, 2)
            st.caption(f"Na místě zatím {celkem:.2f} h. Když část dne děláš hodin
                       "vyber „hodiny i metry" a zadej obojí.")
            rezim = vyber_rezim(u, "odchod_rezim")
            with st.form("odchod"):
                hodiny, metry = 0.0, 0.0
                if rezim in ("hodiny", "hodiny i metry"):
                    vychozi = min(celkem, 24.0) if rezim == "hodiny" else 0.0
                    hodiny = st.number_input(
                        "Hodiny placené hodinově", min_value=0.0, max_value=24.0,
                        value=float(vychozi), step=0.25, key=f"odch_h_{rezim}",
                    )
                if rezim in ("metry", "hodiny i metry"):
                    metry = st.number_input("Metry", min_value=0.0, step=0.5, key
                poz = st.text_input("Poznámka (nepovinné)")
                if st.form_submit_button("🔴 ODCHOD", use_container_width=True):
                    konec = now()
                    celkem = round((konec - start).total_seconds() / 3600, 2)
                    chyba = chyba_vstupu(rezim, hodiny, metry)
                    if chyba:
                        st.error(chyba)
                    elif hodiny > celkem + 0.01:
                        st.error(f"Hodiny nemůžou být víc než čas na místě ({celk
                    else:
                        sh, sm = sazby(u, rezim)
```

```
                    if rezim == "metry":
                        hodiny = celkem  # jen pro informaci, neplatí se
                    run(
                        """UPDATE dochazka SET odchod=%s, hodiny=%s, metry=%s
                                  sazba_hod=%s, sazba_m=%s WHERE id=%s""",
                        (konec.strftime(FMT), hodiny, float(metry), poz, sh,
                    )
                    st.rerun()
        else:
            zid, novy = vyber_mista(uid, "prichod")
            if st.button("🟢 PŘÍCHOD", use_container_width=True, disabled=not (zi
                zid = zakazka_id(zid, novy)
                t = now()
                run(
                    """INSERT INTO dochazka(user_id, zakazka_id, datum, prichod,
                       VALUES(%s,%s,%s,%s,0,0)""",
                    (uid, zid, t.strftime("%Y-%m-%d"), t.strftime(FMT)),
                )
                reset_mista("prichod")
                st.rerun()

        dnes = q(
            """SELECT z.nazev AS "Místo", d.hodiny AS "Hodiny", d.metry AS "Metry
               FROM dochazka d JOIN zakazky z ON z.id=d.zakazka_id
               WHERE d.user_id=%s AND d.datum=%s AND d.hodiny IS NOT NULL
               ORDER BY d.id""",
            (uid, str(now().date())),
        )
        if not dnes.empty:
            st.caption("Dnes zapsáno")
            st.dataframe(dnes, use_container_width=True, hide_index=True)

    with tab2:
        st.caption("Když jsi zapomněl/a dát příchod/odchod, nebo zapisuješ zpětně
                   "Když jsi byl/a v jednom dni na víc místech, zapiš každé zvláš
        zid, novy = vyber_mista(uid, "rucni")
        den = st.date_input("Datum", value=now().date(), key="rucni_den")
        rezim = vyber_rezim(u, "rucni_rezim")
        hodiny, metry = 0.0, 0.0
        if rezim in ("hodiny", "hodiny i metry"):
            hodiny = st.number_input("Hodiny placené hodinově", min_value=0.0, ma
                                     step=0.25, key=f"rucni_h_{rezim}")
        if rezim in ("metry", "hodiny i metry"):
            metry = st.number_input("Metry", min_value=0.0, step=0.5, key=f"rucni
        poz = st.text_input("Poznámka", key="rucni_poz")
        if st.button("Uložit", use_container_width=True, key="rucni_uloz"):
```

```
        if not (zid or novy):
            st.error("Vyber nebo napiš místo.")
        elif chyba_vstupu(rezim, hodiny, metry):
            st.error(chyba_vstupu(rezim, hodiny, metry))
        else:
            zid = zakazka_id(zid, novy)
            sh, sm = sazby(u, rezim)
            run(
                """INSERT INTO dochazka(user_id, zakazka_id, datum, hodiny, m
                   VALUES(%s,%s,%s,%s,%s,%s,%s,%s)""",
                (uid, zid, str(den), float(hodiny), float(metry), poz, sh, sm
            )
            reset_mista("rucni")
            st.success("Uloženo.")

with tab3:
    df = q(
        """SELECT d.id, d.datum AS "Datum", z.nazev AS "Zakázka", d.hodiny AS
                  d.metry AS "Metry", d.schvaleno AS "Schváleno", d.poznamka
           FROM dochazka d JOIN zakazky z ON z.id=d.zakazka_id
           WHERE d.user_id=%s ORDER BY d.datum DESC, d.id DESC LIMIT 100""",
        (uid,),
    )
    if df.empty:
        st.info("Zatím žádné záznamy.")
    else:
        show = df.copy()
        show["Schváleno"] = show["Schváleno"].map({1: "✅", 0: "⏳"})
        st.dataframe(show.drop(columns="id"), use_container_width=True, hide_
        neschv = df[df["Schváleno"] == 0]
        if not neschv.empty:
            with st.expander("Smazat chybný záznam"):
                popis = {
                    int(r.id): f"{r.Datum} – {r.Zakázka} – {r.Hodiny} h"
                    for r in neschv.itertuples()
                }
                sel = st.selectbox("Záznam", list(popis), format_func=lambda
                if st.button("Smazat"):
                    run(
                        "DELETE FROM dochazka WHERE id=%s AND user_id=%s AND
                        (int(sel), uid),
                    )
                    st.rerun()

    zal = q(
        """SELECT datum AS "Datum", castka AS "Záloha (Kč)", poznamka AS "Poz
```

```
                FROM zalohy WHERE user_id=%s AND datum >= %s ORDER BY datum DESC""
            (uid, f"{now().year}-01-01"),
        )
        if not zal.empty:
            st.subheader(f"Moje zálohy {now().year}")
            st.dataframe(zal, use_container_width=True, hide_index=True)

    with tab4:
        zid, novy = vyber_mista(uid, "foto")
        with st.form("foto", clear_on_submit=True):
            files = st.file_uploader(
                "Vyfoť nebo vyber fotky",
                type=["jpg", "jpeg", "png", "webp"],
                accept_multiple_files=True,
            )
            popis = st.text_input("Popis (např. hotová stěna, 2. patro)")
            if st.form_submit_button("📷 Nahrát", use_container_width=True):
                if not files:
                    st.error("Vyber aspoň jednu fotku.")
                elif not (zid or novy):
                    st.error("Vyber nebo napiš místo.")
                else:
                    zid = zakazka_id(zid, novy)
                    for f in files:
                        run(
                            "INSERT INTO fotky(user_id, zakazka_id, datum, popis,
                            (uid, zid, str(now().date()), popis, psycopg2.Binary(
                        )
                    reset_mista("foto")
                    st.success(f"Nahráno fotek: {len(files)}")

        moje = q(
            """SELECT f.datum, f.popis, z.nazev, f.data FROM fotky f
               JOIN zakazky z ON z.id=f.zakazka_id
               WHERE f.user_id=%s ORDER BY f.id DESC LIMIT 6""",
            (uid,),
        )
        if not moje.empty:
            st.caption("Tvoje poslední fotky")
            for r in moje.itertuples():
                st.image(bytes(r.data), caption=f"{r.datum} · {r.nazev}" + (f" ·


# ---------- admin ----------
MESICE = ["led", "úno", "bře", "dub", "kvě", "čvn", "čvc", "srp", "zář", "říj", "
```

```
def kc(x):
    return f"{x:,.0f} Kč".replace(",", " ")


def vydelek(df):
    df = df.copy()
    for col in ("hodiny", "metry", "sazba_hod", "sazba_m"):
        df[col] = pd.to_numeric(df[col], errors="coerce").fillna(0).astype(float)
    df["vydelek"] = df["hodiny"] * df["sazba_hod"] + df["metry"] * df["sazba_m"]
    return df


def zalohy_souctem(od, do):
    return q(
        """SELECT user_id, SUM(castka) AS zalohy FROM zalohy
           WHERE datum BETWEEN %s AND %s GROUP BY user_id""",
        (str(od), str(do)),
    )


def tab_prehled():
    c1, c2 = st.columns(2)
    od = c1.date_input("Od", value=now().date().replace(day=1))
    do = c2.date_input("Do", value=now().date())

    users = q("SELECT id, jmeno FROM users ORDER BY jmeno")
    zakazky = q("SELECT id, nazev FROM zakazky ORDER BY nazev")
    f1, f2 = st.columns(2)
    fu = f1.multiselect("Zaměstnanec", users["jmeno"])
    fz = f2.multiselect("Zakázka", zakazky["nazev"])

    df = q(
        """SELECT d.id, d.user_id, d.datum, u.jmeno, z.nazev, d.hodiny, d.metry,
                  d.sazba_hod, d.sazba_m, d.schvaleno, d.poznamka
           FROM dochazka d
           JOIN users u ON u.id=d.user_id
           JOIN zakazky z ON z.id=d.zakazka_id
           WHERE d.datum BETWEEN %s AND %s
           ORDER BY d.datum DESC, u.jmeno""",
        (str(od), str(do)),
    )
    if fu:
        df = df[df["jmeno"].isin(fu)]
    if fz:
        df = df[df["nazev"].isin(fz)]
```

```
if df.empty:
    st.info("V tomto období nejsou žádné záznamy.")
    return

df = vydelek(df)
zal = zalohy_souctem(od, do)
zal_map = dict(zip(zal["user_id"], zal["zalohy"]))
uids = df["user_id"].unique()
zalohy_celkem = float(sum(zal_map.get(int(i), 0) for i in uids))

m1, m2 = st.columns(2)
m1.metric("Hodiny", f"{df['hodiny'].sum():.1f}")
m2.metric("Metry", f"{df['metry'].sum():.1f}")
m3, m4, m5 = st.columns(3)
m3.metric("Výdělek", kc(df["vydelek"].sum()))
m4.metric("Zálohy", kc(zalohy_celkem))
m5.metric("K výplatě", kc(df["vydelek"].sum() - zalohy_celkem))

st.subheader("Po zaměstnancích")
po_zam = (
    df.groupby(["user_id", "jmeno"])
    .agg(hodiny=("hodiny", "sum"), metry=("metry", "sum"), vydelek=("vydelek"
    .reset_index()
)
po_zam["zalohy"] = po_zam["user_id"].map(lambda i: float(zal_map.get(int(i),
po_zam["k_vyplate"] = po_zam["vydelek"] - po_zam["zalohy"]
po_zam = po_zam.drop(columns="user_id").round(2)
st.dataframe(po_zam, use_container_width=True, hide_index=True)

st.subheader("Po zakázkách")
po_zak = df.groupby("nazev").agg(hodiny=("hodiny", "sum"), metry=("metry", "s
st.dataframe(po_zak, use_container_width=True, hide_index=True)

st.subheader("Jednotlivé záznamy")
st.caption("Můžeš opravit hodiny, metry, sazby, poznámku a zaškrtnout schvále
edit = df[["id", "datum", "jmeno", "nazev", "hodiny", "metry", "sazba_hod", "
edit["schvaleno"] = edit["schvaleno"].astype(bool)
edited = st.data_editor(
    edit,
    hide_index=True,
    use_container_width=True,
    disabled=["id", "datum", "jmeno", "nazev"],
    column_config={
        "schvaleno": st.column_config.CheckboxColumn("Schváleno"),
        "sazba_hod": st.column_config.NumberColumn("Kč/h", min_value=0, step=
```

```
            "sazba_m": st.column_config.NumberColumn("Kč/m", min_value=0, step=10
            "id": None,
        },
        key="edit_dochazka",
    )
    if st.button("💾 Uložit změny", type="primary"):
        run_many([
            (
                """UPDATE dochazka SET hodiny=%s, metry=%s, sazba_hod=%s, sazba_m
                          poznamka=%s, schvaleno=%s WHERE id=%s""",
                (nn(r.hodiny), nn(r.metry), nn(r.sazba_hod), nn(r.sazba_m), nn(r.
            )
            for r in edited.itertuples()
        ])
        st.success("Uloženo.")
        st.rerun()

    buf = io.BytesIO()
    with pd.ExcelWriter(buf, engine="openpyxl") as xw:
        df.drop(columns="user_id").to_excel(xw, sheet_name="Záznamy", index=False
        po_zam.to_excel(xw, sheet_name="Po zaměstnancích", index=False)
        po_zak.to_excel(xw, sheet_name="Po zakázkách", index=False)
    st.download_button(
        "📥 Export do Excelu",
        buf.getvalue(),
        file_name=f"dochazka_{od}_{do}.xlsx",
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    )


def tab_detail():
    users = q("SELECT id, jmeno FROM users WHERE aktivni=1 ORDER BY jmeno")
    if users.empty:
        st.info("Zatím nejsou žádní zaměstnanci.")
        return

    t = now().date()
    mesice, y, m = [], t.year, t.month
    for _ in range(12):
        mesice.append((y, m))
        m -= 1
        if m == 0:
            m, y = 12, y - 1

    c1, c2 = st.columns(2)
    jmeno = c1.selectbox("Zaměstnanec", users["jmeno"], key="det_user")
```

```
ym = c2.selectbox("Měsíc", mesice, format_func=lambda x: f"{x[1]:02d}/{x[0]}"
uid = int(users.loc[users["jmeno"] == jmeno, "id"].iloc[0])
od = date(ym[0], ym[1], 1)
do = date(ym[0], ym[1], calendar.monthrange(ym[0], ym[1])[1])

df = q(
    """SELECT d.id, d.datum, z.nazev, d.prichod, d.odchod, d.hodiny, d.metry,
              d.sazba_hod, d.sazba_m, d.schvaleno, d.poznamka
       FROM dochazka d
       JOIN zakazky z ON z.id=d.zakazka_id
       WHERE d.user_id=%s AND d.datum BETWEEN %s AND %s
       ORDER BY d.datum, d.id""",
    (uid, str(od), str(do)),
)
zal = q(
    "SELECT datum, castka, poznamka FROM zalohy WHERE user_id=%s AND datum BE
    (uid, str(od), str(do)),
)
zalohy = float(zal["castka"].sum()) if not zal.empty else 0.0

if df.empty and zal.empty:
    st.info(f"{jmeno} nemá v tomto měsíci žádné záznamy.")
    return

df = vydelek(df) if not df.empty else df
hodiny = float(df["hodiny"].sum()) if not df.empty else 0.0
metry = float(df["metry"].sum()) if not df.empty else 0.0
vyd = float(df["vydelek"].sum()) if not df.empty else 0.0

m1, m2, m3 = st.columns(3)
m1.metric("Hodiny", f"{hodiny:.1f}")
m2.metric("Metry", f"{metry:.1f}")
m3.metric("Dnů", df["datum"].nunique() if not df.empty else 0)
v1, v2, v3 = st.columns(3)
v1.metric("Výdělek", kc(vyd))
v2.metric("Zálohy", kc(zalohy))
v3.metric("K výplatě", kc(vyd - zalohy))

den = pd.DataFrame()
po_zak = pd.DataFrame()
if not df.empty:
    st.subheader("Součet po zakázkách")
    po_zak = (
        df.groupby("nazev")
        .agg(dnu=("datum", "nunique"), hodiny=("hodiny", "sum"), metry=("metr
        .reset_index()
```

```
        .rename(columns={"nazev": "Zakázka", "dnu": "Dnů", "hodiny": "Hodiny"
        .round(2)
    )
    st.dataframe(po_zak, use_container_width=True, hide_index=True)

    st.subheader("Den po dni")
    den = pd.DataFrame(
        {
            "Den": df["datum"].map(lambda x: f"{CZ_DNY[date.fromisoformat(x).
            "Zakázka": df["nazev"],
            "Příchod": df["prichod"].map(lambda x: x[11:16] if isinstance(x,
            "Odchod": df["odchod"].map(lambda x: x[11:16] if isinstance(x, st
            "Hodiny": df["hodiny"].round(2),
            "Metry": df["metry"].round(2),
            "Výdělek": df["vydelek"].round(0),
            "Schváleno": df["schvaleno"].map({1: "✅", 0: "⏳"}),
            "Poznámka": df["poznamka"].fillna(""),
        }
    )
    st.dataframe(den, use_container_width=True, hide_index=True)

if not zal.empty:
    st.subheader("Zálohy v měsíci")
    st.dataframe(
        zal.rename(columns={"datum": "Datum", "castka": "Částka (Kč)", "pozna
        use_container_width=True, hide_index=True,
    )

b1, b2 = st.columns(2)
if b1.button("✅ Schválit celý měsíc", use_container_width=True):
    run(
        """UPDATE dochazka SET schvaleno=1
           WHERE user_id=%s AND datum BETWEEN %s AND %s AND (odchod IS NOT NU
        (uid, str(od), str(do)),
    )
    st.rerun()

buf = io.BytesIO()
with pd.ExcelWriter(buf, engine="openpyxl") as xw:
    if not den.empty:
        den.to_excel(xw, sheet_name="Den po dni", index=False)
        po_zak.to_excel(xw, sheet_name="Po zakázkách", index=False)
    pd.DataFrame(
        {"Položka": ["Výdělek", "Zálohy", "K výplatě"], "Kč": [round(vyd, 2),
    ).to_excel(xw, sheet_name="Výplata", index=False)
b2.download_button(
```

```
        "📥 Excel",
        buf.getvalue(),
        file_name=f"{jmeno}_{ym[0]}-{ym[1]:02d}.xlsx",
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        use_container_width=True,
    )
    st.caption("Chybný záznam oprav v záložce Přehled (úprava hodin, metrů, sazeb


def tab_zalohy():
    users = q("SELECT id, jmeno FROM users WHERE aktivni=1 AND role='zamestnanec'
    if users.empty:
        st.info("Nejdřív přidej zaměstnance.")
        return

    with st.form("nova_zaloha", clear_on_submit=True):
        st.subheader("Nová záloha")
        jmeno = st.selectbox("Zaměstnanec", users["jmeno"])
        den = st.date_input("Datum", value=now().date())
        castka = st.number_input("Částka (Kč)", min_value=0.0, step=100.0)
        poz = st.text_input("Poznámka")
        if st.form_submit_button("Přidat zálohu", use_container_width=True):
            if castka <= 0:
                st.error("Zadej částku.")
            else:
                uid = int(users.loc[users["jmeno"] == jmeno, "id"].iloc[0])
                run(
                    "INSERT INTO zalohy(user_id, datum, castka, poznamka) VALUES(
                    (uid, str(den), float(castka), poz),
                )
                st.rerun()

    rok = int(st.number_input("Rok", min_value=2020, max_value=2100, value=now().
    df = q(
        """SELECT z.id, z.datum, u.jmeno, z.castka, z.poznamka FROM zalohy z
           JOIN users u ON u.id=z.user_id
           WHERE z.datum BETWEEN %s AND %s ORDER BY z.datum DESC""",
        (f"{rok}-01-01", f"{rok}-12-31"),
    )

    st.subheader(f"Zálohy {rok} po měsících")
    tab = pd.DataFrame(0.0, index=list(users["jmeno"]), columns=MESICE)
    if not df.empty:
        for r in df.itertuples():
            if r.jmeno in tab.index:
                tab.loc[r.jmeno, MESICE[int(r.datum[5:7]) - 1]] += float(r.castka
```

```
    tab["Celkem"] = tab.sum(axis=1)
    tab.index.name = "Zaměstnanec"
    st.dataframe(tab.round(0).reset_index(), use_container_width=True, hide_index

    if not df.empty:
        with st.expander("Seznam záloh / smazat zálohu"):
            st.dataframe(
                df.drop(columns="id").rename(columns={"datum": "Datum", "jmeno":
                use_container_width=True, hide_index=True,
            )
            popis = {int(r.id): f"{r.datum} – {r.jmeno} – {r.castka:.0f} Kč" for
            sel = st.selectbox("Záloha ke smazání", list(popis), format_func=lamb
            if st.button("Smazat zálohu"):
                run("DELETE FROM zalohy WHERE id=%s", (int(sel),))
                st.rerun()


def tab_fotky_admin():
    zak = q("SELECT id, nazev FROM zakazky ORDER BY nazev")
    if zak.empty:
        st.info("Zatím nejsou žádné zakázky.")
        return
    nazev = st.selectbox("Zakázka", zak["nazev"], key="adm_foto_zak")
    zid = int(zak.loc[zak["nazev"] == nazev, "id"].iloc[0])
    limit = st.number_input("Kolik posledních fotek zobrazit", min_value=2, max_v
    fot = q(
        """SELECT f.id, f.datum, f.popis, u.jmeno, f.data FROM fotky f
           JOIN users u ON u.id=f.user_id
           WHERE f.zakazka_id=%s ORDER BY f.id DESC LIMIT %s""",
        (zid, int(limit)),
    )
    if fot.empty:
        st.info("K této zakázce zatím nejsou žádné fotky.")
        return
    cols = st.columns(2)
    for i, r in enumerate(fot.itertuples()):
        with cols[i % 2]:
            st.image(bytes(r.data), caption=f"{r.datum} · {r.jmeno}" + (f" · {r.p
            if st.button("Smazat", key=f"delfoto_{r.id}"):
                run("DELETE FROM fotky WHERE id=%s", (int(r.id),))
                st.rerun()


def tab_zakazky():
    n_nove = one("SELECT COUNT(*) AS n FROM zakazky WHERE nova=1")["n"]
    if n_nove:
```

```
    st.warning(f"Nové zakázky od zaměstnanců: {n_nove}. Doplň klienta a odškr
               "Pokud je to překlep, sluč ji s existující níže.")

with st.form("nova_zakazka", clear_on_submit=True):
    st.subheader("Nová zakázka")
    nazev = st.text_input("Název (např. Rekonstrukce Novák, Ostrava)")
    klient = st.text_input("Klient (nepovinné)")
    if st.form_submit_button("Přidat", use_container_width=True):
        n = norm(nazev)
        if not n:
            st.error("Zadej název.")
        elif one("SELECT id FROM zakazky WHERE nazev_norm=%s", (n,)):
            st.error("Taková zakázka už existuje.")
        else:
            try:
                run(
                    "INSERT INTO zakazky(nazev, nazev_norm, klient, sazba, no
                    (cisty_nazev(nazev), n, klient),
                )
                st.rerun()
            except psycopg2.IntegrityError:
                st.error("Taková zakázka už existuje.")

st.subheader("Seznam zakázek")
df = q(
    """SELECT z.id, z.nazev, z.klient, z.aktivni, z.nova,
              (SELECT COUNT(*) FROM dochazka d WHERE d.zakazka_id=z.id) AS za
       FROM zakazky z ORDER BY z.nova DESC, z.aktivni DESC, z.nazev"""
)
if df.empty:
    return
df["aktivni"] = df["aktivni"].astype(bool)
df["nova"] = df["nova"].astype(bool)
edited = st.data_editor(
    df,
    hide_index=True,
    use_container_width=True,
    disabled=["id", "nazev", "zaznamu"],
    column_config={
        "id": None,
        "nazev": "Zakázka",
        "klient": "Klient",
        "aktivni": st.column_config.CheckboxColumn("Aktivní"),
        "nova": st.column_config.CheckboxColumn("Nová"),
        "zaznamu": "Záznamů",
    },
```

```
    key="edit_zakazky",
)
if st.button("💾 Uložit zakázky"):
    run_many([
        (
            "UPDATE zakazky SET klient=%s, aktivni=%s, nova=%s WHERE id=%s",
            (nn(r.klient), int(r.aktivni), int(r.nova), int(r.id)),
        )
        for r in edited.itertuples()
    ])
    st.success("Uloženo.")
    st.rerun()

popis = {int(r.id): r.nazev for r in df.itertuples()}

with st.expander("Sloučit duplicitní zakázky (např. překlep)"):
    zdroj = st.selectbox("Tuto zakázku zrušit", list(popis), format_func=lamb
    cil = st.selectbox("A záznamy a fotky převést sem", list(popis), format_f
    if st.button("Sloučit"):
        if zdroj == cil:
            st.error("Vyber dvě různé zakázky.")
        else:
            run_many([
                ("UPDATE dochazka SET zakazka_id=%s WHERE zakazka_id=%s", (in
                ("UPDATE fotky SET zakazka_id=%s WHERE zakazka_id=%s", (int(c
                ("DELETE FROM prirazeni WHERE zakazka_id=%s", (int(zdroj),)),
                ("DELETE FROM zakazky WHERE id=%s", (int(zdroj),)),
            ])
            st.success("Sloučeno.")
            st.rerun()

with st.expander("Přejmenovat zakázku"):
    kdo = st.selectbox("Zakázka", list(popis), format_func=lambda i: popis[i]
    novy = st.text_input("Nový název", key="pr_novy")
    if st.button("Přejmenovat"):
        n = norm(novy)
        existuje = one("SELECT id FROM zakazky WHERE nazev_norm=%s AND id<>%s
        if not n:
            st.error("Zadej název.")
        elif existuje:
            st.error("Zakázka s tímto názvem už existuje. Použij sloučení.")
        else:
            run("UPDATE zakazky SET nazev=%s, nazev_norm=%s WHERE id=%s", (ci
            st.rerun()
```

```
def tab_zamestnanci():
    with st.form("novy_user", clear_on_submit=True):
        st.subheader("Nový zaměstnanec")
        j = st.text_input("Jméno (pod ním se bude přihlašovat)")
        h = st.text_input("Heslo (min. 6 znaků)", type="password")
        role = st.selectbox("Role", ["zamestnanec", "admin"])
        c1, c2 = st.columns(2)
        sh = c1.number_input("Sazba Kč/hodinu", min_value=0.0, step=10.0)
        sm = c2.number_input("Sazba Kč/metr", min_value=0.0, step=10.0)
        st.caption("Co se nepoužívá, nech 0. Můžeš vyplnit obě sazby.")
        if st.form_submit_button("Přidat", use_container_width=True):
            if not j.strip() or len(h) < 6:
                st.error("Zadej jméno a heslo (min. 6 znaků).")
            else:
                try:
                    create_user(j, h, role, sh, sm)
                    st.rerun()
                except psycopg2.IntegrityError:
                    st.error("Toto jméno už existuje.")

    st.subheader("Seznam a sazby")
    df = q("SELECT id, jmeno, role, sazba_hod, sazba_m, aktivni FROM users ORDER
    df["aktivni"] = df["aktivni"].astype(bool)
    bez = df[(df["role"] == "zamestnanec") & (df["aktivni"]) & (df["sazba_hod"] =
    if not bez.empty:
        st.warning("Bez sazby (výdělek se bude počítat jako 0): " + ", ".join(bez
    edited = st.data_editor(
        df,
        hide_index=True,
        use_container_width=True,
        disabled=["id", "jmeno", "role"],
        column_config={
            "id": None,
            "jmeno": "Jméno",
            "role": "Role",
            "sazba_hod": st.column_config.NumberColumn("Kč/h", min_value=0, step=
            "sazba_m": st.column_config.NumberColumn("Kč/m", min_value=0, step=10
            "aktivni": st.column_config.CheckboxColumn("Aktivní"),
        },
        key="edit_users",
    )
    prepocet = st.checkbox("Změněné sazby přepsat i do dosud neschválených záznam
    st.caption("Schválené záznamy se nikdy nemění. Nové záznamy berou vždy aktuál
    if st.button("💾 Uložit zaměstnance"):
        orig = df.set_index("id")
        stmts = []
```

```
        for r in edited.itertuples():
            uid = int(r.id)
            sh = float(nn(r.sazba_hod) or 0)
            sm = float(nn(r.sazba_m) or 0)
            aktivni = True if uid == st.session_state.user["id"] else bool(r.akti
            stmts.append((
                "UPDATE users SET aktivni=%s, sazba_hod=%s, sazba_m=%s WHERE id=%
                (int(aktivni), sh, sm, uid),
            ))
            if prepocet:
                if sh != float(orig.loc[uid, "sazba_hod"]):
                    stmts.append((
                        "UPDATE dochazka SET sazba_hod=%s WHERE user_id=%s AND sc
                        (sh, uid),
                    ))
                if sm != float(orig.loc[uid, "sazba_m"]):
                    stmts.append((
                        "UPDATE dochazka SET sazba_m=%s WHERE user_id=%s AND schv
                        (sm, uid),
                    ))
        run_many(stmts)
        st.success("Uloženo.")
        st.rerun()

    with st.expander("Změnit heslo zaměstnanci"):
        with st.form("reset_pw", clear_on_submit=True):
            kdo = st.selectbox("Kdo", df["jmeno"])
            nove = st.text_input("Nové heslo", type="password")
            if st.form_submit_button("Změnit"):
                if len(nove) < 6:
                    st.error("Min. 6 znaků.")
                else:
                    uid = int(df.loc[df["jmeno"] == kdo, "id"].iloc[0])
                    set_password(uid, nove)
                    st.success("Heslo změněno.")


def page_admin():
    n_nove = one("SELECT COUNT(*) AS n FROM zakazky WHERE nova=1 AND aktivni=1")[
    zak_label = f"🏗️ Zakázky ({n_nove} nové)" if n_nove else "🏗️ Zakázky"
    t1, t2, t3, t4, t5, t6 = st.tabs(
        ["👤 Zaměstnanec", "📊 Přehled", "💰 Zálohy", "📷 Fotky", zak_label, "👷
    )
    with t1:
        tab_detail()
    with t2:
```

```
            tab_prehled()
        with t3:
            tab_zalohy()
        with t4:
            tab_fotky_admin()
        with t5:
            tab_zakazky()
        with t6:
            tab_zamestnanci()


# ---------- hlavní ----------
def main():
    if not db_url():
        st.error("Chybí nastavení DATABASE_URL. Viz návod.")
        st.stop()
    init_db()
    user = st.session_state.get("user")
    if not user:
        page_login()
        return

    top1, top2 = st.columns([3, 1])
    top1.markdown(f"### 🏗️ {user['jmeno']}")
    if top2.button("Odhlásit"):
        st.session_state.clear()
        st.rerun()

    if user["role"] == "admin":
        page_admin()
    else:
        page_zamestnanec(user)


main()
```